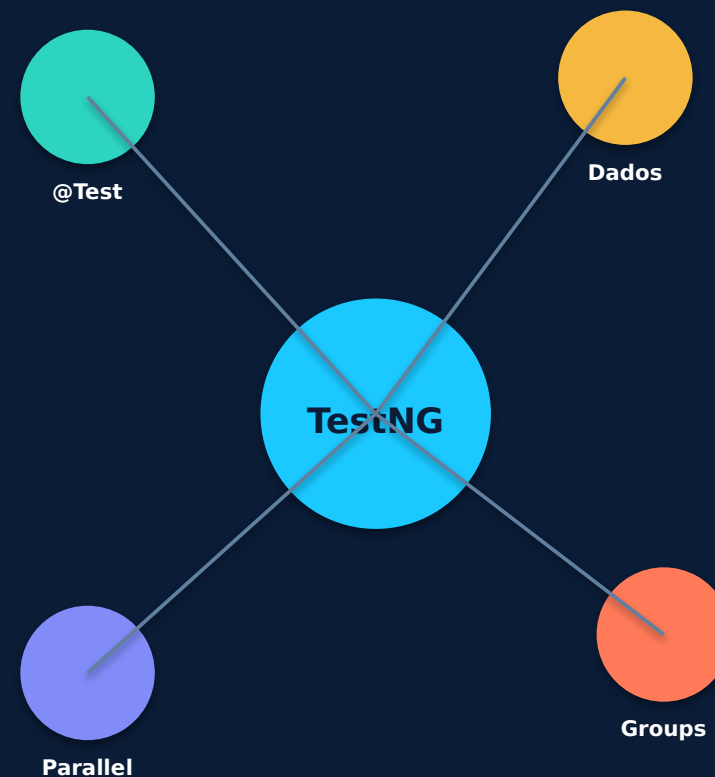


O maestro dos testes

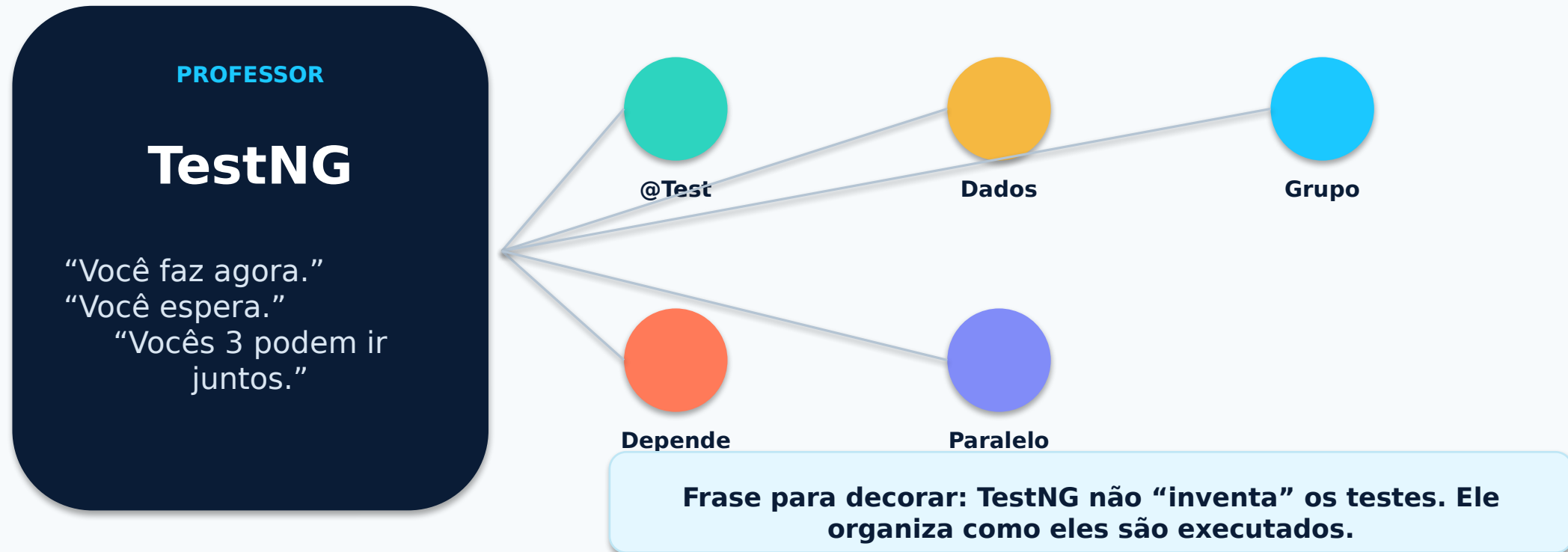
Como organizar, repetir e acelerar testes Java sem virar bagunça

A1-01 • Seminário prático



Pense numa sala de aula com muitos alunos...

Cada “aluno” é um teste. O TestNG é o professor que organiza a turma.



O que o professor vai receber?

Um pacote pequeno, mas completo: explicação + código + prova de que funciona.



PDF da apresentação

o que você explica em 20 min



README.md

manual para entender e executar



Código Java

o “brinquedo” que vamos testar



Testes TestNG

@Test, dados, grupos e dependências



testng.xml

onde controlamos o paralelismo



PR no GitHub

o link final da entrega

O que é TestNG em uma frase?

“Um organizador de testes Java.”

@Test

marca o que deve rodar

@Before/After

prepara e limpa

@DataProvider

repete com vários dados

Groups

separa a suíte

Dependencies

cria ordem quando
precisa

Parallel

roda coisas juntas

Versão usada no projeto: TestNG 7.12.0 • Java 21 • Maven

Um mini sistema de transferência

Pense num caixa eletrônico muito simples.

Regras do “mini banco”

Mínimo R\$ 10

Máximo R\$ 10.000

Saldo não pode ficar negativo

Pergunta do teste: “Essa transferência é válida?”

```
public boolean isValid(double saldo, double valor) {  
    return valor >= 10  
        && valor <= 10000  
        && valor <= saldo;  
}
```

É um exemplo pequeno de propósito: fácil de entender, bom para demonstrar TestNG.

@Test = fazer uma pergunta e conferir a resposta

```
@Test
public void deveAceitarTransferenciaValida() {
    boolean resultado = service.isValid(5000,
    1000);
    Assert.assertTrue(resultado);
}
```

1

@Test

“rode este método”

2

Executa

faz a pergunta ao sistema

3

Assert

confere se a resposta está certa

É como perguntar “2 + 2 = 4?” e conferir “sim”.

Antes, durante e depois do teste

Imagine arrumar a mesa antes da brincadeira e guardar tudo quando terminar.



Por que isso importa? Cada teste começa “limpo”, sem herdar bagunça do teste anterior.

DataProvider = o mesmo exercício com várias respostas possíveis

Caixa de exemplos



O TestNG pega um valor por vez e repete o MESMO teste.

```
@DataProvider(name = "limites")
public Object[][] dados() {
    return new Object[][] {
        {9, false}, {10, true}, {11, true},
        {9999, true}, {10000, true}, {10001, false}
    };
}
```

BVA = testar a beirada da piscina

Erros gostam de morar perto dos limites.

Mínimo = 10

Máximo = 10.000



Pense numa porta: queremos testar um pouco antes, exatamente na porta e um pouco depois.

ECP = escolher 1 representante de cada “família”

Muito pequeno

< 10

ex.: 5

Válido

10 a 10.000

ex.: 500

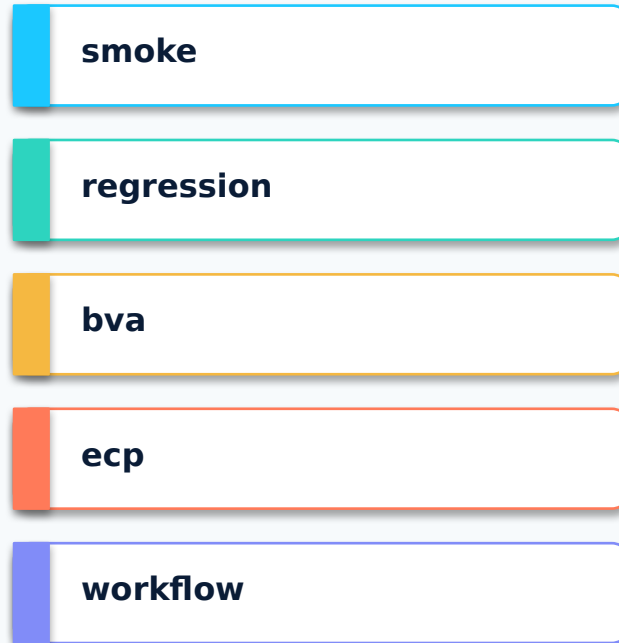
Muito grande

> 10.000

ex.: 15.000

Não precisamos testar todos os números do mundo. Testamos bons representantes.

Groups = gavetas para separar os testes



`mvn -Dgroups=smoke test`

“Rode só os testes essenciais.”

Ótimo para CI/CD: rápido agora, suíte completa depois.

Dependencies = dominós: uma etapa depende da anterior



```
@Test(dependsOnMethods = "authenticate")  
public void executeTransfer() { ... }
```

Cuidado: dependência demais deixa os testes grudados uns nos outros.

Parallel = abrir várias filas ao mesmo tempo

Fila 1

cenário A

Fila 2

cenário B

Fila 3

cenário C



```
<suite parallel="methods" thread-  
count="3">
```

3 threads = até 3 métodos rodando juntos

**Mas só funciona bem quando
cada teste é independente.**

A validação encontrou um problema de verdade

Nossa primeira versão tinha estado compartilhado durante o paralelismo.

ANTES

```
private TransferService service;  
  
@BeforeMethod  
void setUp() { service = new TransferService(); }
```

Duas threads podiam mexer na mesma referência.

DEPOIS

```
private final ThreadLocal<TransferService> service =  
    new ThreadLocal<>();  
  
@BeforeMethod  
void setUp() { service.set(new TransferService()); }
```

Cada thread ganhou sua própria instância.

Lição: paralelismo não é só “ficar mais rápido”. Também exige thread safety.

Demo ao vivo em 5 minutos

A ordem abaixo evita que você se perca.

1	Mostrar TransferService	"Este é o sistema."	40s
2	Mostrar @Test + Assert	"Este é o teste básico."	50s
3	Mostrar DataProvider + BVA	"Mesmo teste, vários dados."	70s
4	Mostrar groups + dependências	"Organizo e crio ordem."	55s
5	Rodar paralelo	<code>mvn -Psuite test</code>	85s

Plano B: se o terminal falhar, explique o código e mostre o resultado validado no GitHub Actions.

TestNG x JUnit moderno

Não é “bom x ruim”. É escolher a ferramenta que combina com a suíte.

Recurso	TestNG	JUnit
Teste parametrizado	@DataProvider	@ParameterizedTest
Agrupamento	groups	@Tag
Dependências	nativo	sem equivalente direto simples
Paralelismo	sim	sim
Suíte XML	forte	abordagem diferente

Minha conclusão: TestNG brilha quando organização e orquestração da suíte viram parte importante do problema.

O que realmente aprendemos?

TÉCNICA

BVA + ECP

escolhem bons casos

FRAMEWORK

TestNG

executa e organiza

ESCALA

Groups + Parallel

faz a suíte crescer

TestNG não decide O QUE testar. Ele ajuda a decidir COMO executar bem.

Obrigado.

Perguntas que podem aparecer

Use só se o professor perguntar.

TestNG substitui Selenium?

Não. Selenium controla o navegador; TestNG organiza os testes.

TestNG é caixa-preta ou caixa-branca?

Pode ser usado nos dois; ele é framework, não técnica.

Por que DataProvider?

Para repetir o mesmo teste com vários conjuntos de dados.

Dependência é sempre boa?

Não. Use com cuidado para não criar testes frágeis.

Qual risco do parallel?

Estado compartilhado e race conditions.